SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

Disaster Relief Resource & Camp Management System

Program	B.Tech AI & ML
Section	Leopards
Submitted To	Dr. C. Nagaraju
Date of Submission	13-08-2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name
25WU0102213	D. Pragnesh Reddy

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Dr. C. Nagaraju, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled **“Disaster Relief Resource & Camp Management System”**, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature

Date : 13-08-2026

Place : [Place]

Name: D. Pragnesh Reddy

Roll Number: 25WU0102213

Program: B.Tech AI & ML

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	10
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	11
1.6 Entity-Relationship Diagram (ERD)	12
1.7 Entity and Relationship Summary	13
2. LOGICAL DATABASE DESIGN	13
2.1 Relational Schema	13
2.2 Keys and Integrity Constraints	14
2.3 Functional Dependencies	16
2.4 Normalization up to Third Normal Form (3NF)	16
2.5 Data Dictionary	17
2.6 Implementation of Logical Design: DDL and Sample Data	19
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	21
3.1 Application Overview	21
3.2 Development Environment and Database Connectivity	22
3.3 Application Modules and Database Mapping	22
3.4 CRUD Operations	22
3.5 Business Rules, Validation and Error Handling	23
3.6 Reports and Database Operations	23
3.7 Application Screens	24
3.8 Testing and Results	26
4. CONCLUSION	26
4.1 Limitations and Future Enhancements	27
5. REFERENCES	27
6. APPENDICES	28

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	11
Table 4	Entity and Relationship Summary	13
Table 5	Relational Schema	13
Table 6	Keys and Integrity Constraints	14
Table 7	Normalization Status up to 3NF	17
Table 8	Data Dictionary	17
Table 9	Database Implementation Environment	20
Table 10	Application Development Environment	22
Table 11	Application Module-to-Database Mapping	22
Table 12	CRUD Operations	22
Table 13	Business Rules, Validation and Error Handling	23
Table 14	Reports and Database Operations	23
Table 15	Test Cases	26

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	Family registration, need entry, donation and constraint checks	24
Figure 3	Distribution transactions with business-rule validation	24
Figure 4	Report: camp occupancy, unmet needs and stock alerts	25
Figure 5	Report: audit trail, volunteer workload and families without aid	25

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FD	Functional Dependency
FK	Foreign Key
NGO	Non-Governmental Organization
PBL	Project-Based Learning
PK	Primary Key
SQL	Structured Query Language

ABSTRACT

When floods and cyclones displace families, the aid that reaches them is frequently tracked on separate paper lists. Such records allow the same family to be served twice, leave others unserved, strand surplus stock in one place while another camp runs short, and give nobody a trail showing who released what. This work presents a relational database, with a Python program on top of it, built to run relief camps and warehouses spread over several locations. Its aim is to stop repeat entitlements, share supplies fairly and keep every stock movement and volunteer duty on record. Fourteen entities were modelled in an entity-relationship diagram and mapped to third normal form tables in SQLite, protected by key, unique, check and default constraints plus triggers that apply the business rules. The program handles family registration, needs, donations, distributions, volunteer postings and medical referrals, and prints reports built from joins, grouping and subqueries. Thirteen tests confirm that duplicate needs, excess allocation, repeat entitlement, shortage of stock and broken references are refused, while valid transactions update stock and need status correctly. The outcome is a reliable, traceable basis for coordinating relief at camp level.

KEYWORDS

Keywords: Disaster Relief, Camp Management, Relational Database, ER Diagram, Normalization, Python and SQLite

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

Here the report states the problem being solved, what the project sets out to do, its limits, the planned solution, the required functions and the final ER diagram for the Disaster Relief Resource & Camp Management System.

1.1 Problem Statement

Floods and cyclones force large numbers of families into relief camps, where they rely on regular deliveries of food, drinking water, shelter material and medicines. Humanitarian standards expect this help to be fair and accountable [1]. On the ground, however, camp coordinators, warehouse managers, donors, non-governmental organizations (NGOs) and volunteers usually maintain their own separate paper lists. Details about families, needs, stock, donations and volunteer duties end up scattered, and there is no single reliable source of truth.

Four problems follow from this. A family can be given the same items repeatedly while another gets none, and a paper process has no way to notice it. Because stock releases and volunteer postings are not recorded, nobody can audit them or be held accountable. Supplies cannot be matched with needs across camps and warehouses, so surpluses and shortages exist side by side, and shortages are spotted late. The situation calls for a database whose rules are enforced automatically, so that each family, need, item, donation and distribution is recorded once, linked properly and checked before it is accepted.

1.2 Objectives

- Build a normalized (third normal form) relational database of fourteen entities: disasters, locations, camps, warehouses, families, persons, volunteers, relief items, stock, donations, needs, distributions, assignments and referrals.
- Apply constraints and triggers so that the database itself refuses repeat entitlements, over-allocation and distributions larger than the available stock.
- Create a Python application for adding, viewing, changing and carefully removing records through parameterised queries and transactions.

- Record each distribution with its family, item, source warehouse, quantity, date and volunteer so that it can be audited.
- Produce six reports (camp occupancy, unmet needs, stock alerts, audit trail, volunteer workload, families without aid) using joins, aggregation and subqueries.

1.3 Scope and Assumptions

Table 1 sets out what the system includes, what it leaves out on purpose, and the assumptions made.

Table 1. Scope and Assumptions

Item	Description
In Scope	Disasters, locations, camps and warehouses; families and members; relief items, warehouse stock and donations; family needs and distributions; volunteer postings; medical referrals; and operational reports.
Out of Scope / Exclusions	Live GPS tracking, donation accounting, login and role-based security, links to government systems, and a production web or mobile front end (only a browser prototype is included to demonstrate the rules).
Assumptions	Each family stays in one camp at a time; a family has at most one need record per item, storing its total entitlement; each distribution takes from one stock row of one warehouse; quantities are whole numbers; the data is illustrative; the database is a single-user SQLite file.

1.4 Proposed Solution Strategy

Table 2 describes the planned solution: its users, modules, workflow, database rules and outputs.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Camp coordinators (register families and needs); warehouse managers (stock and donations); volunteers (record distributions); relief administrators (review reports).
Major Application Modules	Camp and family registration; needs; stock and donations; distribution; volunteer assignment; medical referrals; reports.
Proposed Workflow	A disaster is declared and its locations, camps and warehouses are created; families and members are registered and their needs recorded; donations add to warehouse stock; a volunteer issues items against a need from a stock row; triggers check the request and update stock and need status; administrators then review the reports.
Important Business Rules	A family has one need per item; a distribution may not exceed the remaining need or the stock on hand; a fulfilled need cannot be served again; the stock item must be the item needed; stock cannot go negative; a camp with registered families cannot be deleted.
Expected Reports / Outputs	Camp occupancy; unmet needs by camp; total stock with low-stock flag; distribution audit trail; volunteer workload; families that have received no aid.

1.5 Functional Requirements

Table 3 gives the functional requirements that follow from the objectives.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Register families and their members in a camp together with contact details.	High
FR-02	Record a family's need for a relief item and block a second need for the same item.	High
FR-03	Keep stock by item and warehouse, and show total stock with a low-stock flag.	High
FR-04	Record donations and raise the matching warehouse stock automatically.	High
FR-05	Record a distribution against a need from a chosen stock row, noting the volunteer.	High
FR-06	Refuse repeat entitlement, over-allocation, insufficient stock and item mismatch.	High
FR-07	Post volunteers to camps with a role and duty period.	Medium
FR-08	Record medical referrals for registered persons.	Medium
FR-09	Update family contact details and prevent deletion of records still in use.	Medium
FR-10	Produce the six operational reports listed in Table 2.	High

1.6 Entity-Relationship Diagram (ERD)

Figure 1 is the approved ER diagram (rotated 90° to fill the A4 page; turn the page clockwise to read it). It uses crow's-foot notation, where a tee marks the “one” end and a crow's foot the “many” end of a relationship. Primary keys are in bold and foreign keys in italics. Every relationship is one-to-many, and the many side takes part fully because its foreign key cannot be null.

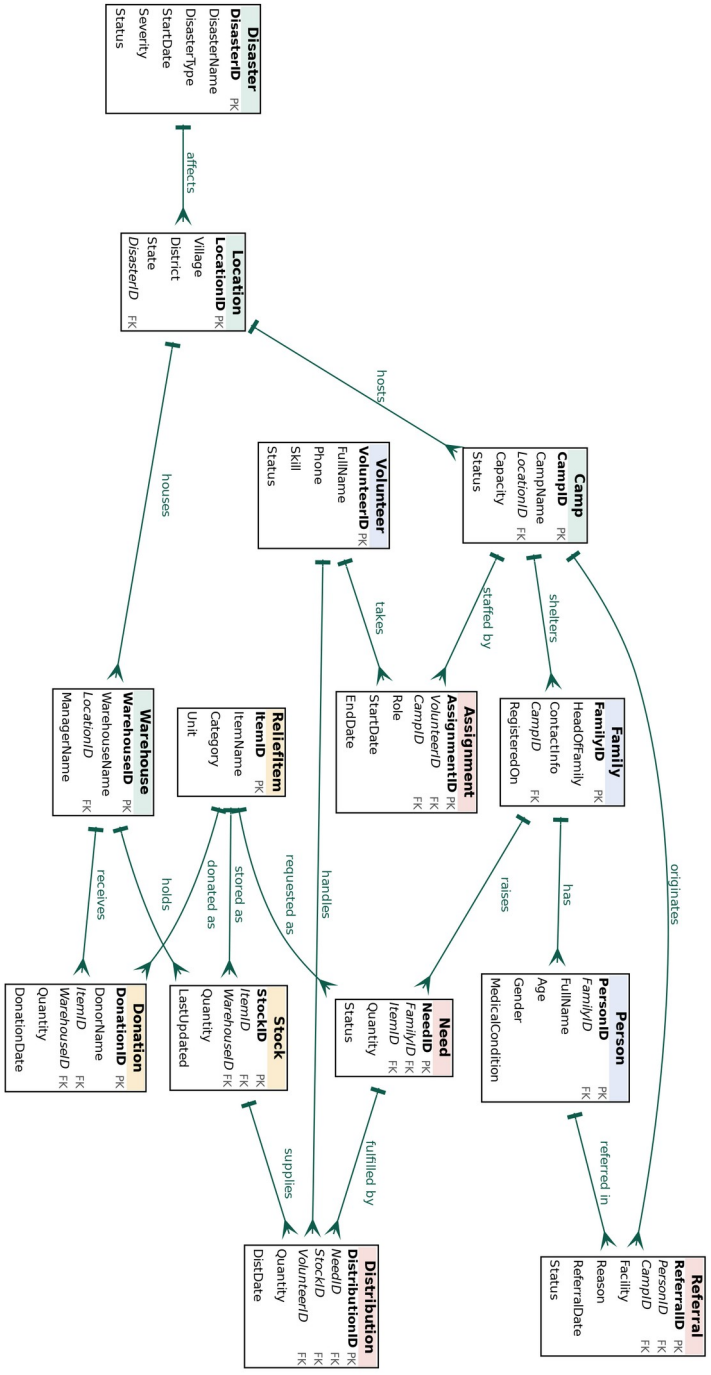


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 lists the fourteen entities with their main attributes, cardinalities and purpose.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Disaster	DisasterID, DisasterName, DisasterType, StartDate, Severity, Status. 1:N with Location.	Disaster event that triggers the relief operation.
Location	LocationID, Village, District, State, DisasterID. 1:N with Camp and Warehouse.	Affected place; hosts camps and warehouses.
Camp	CampID, CampName, LocationID, Capacity, Status. 1:N with Family, Assignment, Referral.	Relief camp that shelters families.
Warehouse	WarehouseID, WarehouseName, LocationID, ManagerName. 1:N with Stock, Donation.	Storage point for relief items.
Family	FamilyID, HeadOfFamily, ContactInfo, CampID. 1:N with Person, Need.	Unit that receives aid; one camp at a time.
Person	PersonID, FamilyID, FullName, Age, Gender, MedicalCondition. 1:N with Referral.	Individual member of a family.
Volunteer	VolunteerID, FullName, Phone, Skill, Status. 1:N with Assignment, Distribution.	Person who staffs camps and hands out items.
ReliefItem	ItemID, ItemName, Category, Unit. 1:N with Stock, Donation, Need.	Catalogue of items (rice, water, blankets, etc.).
Stock	StockID, ItemID, WarehouseID, Quantity, LastUpdated. 1:N with Distribution.	Quantity of one item held in one warehouse.
Donation	DonationID, DonorName, ItemID, WarehouseID, Quantity, DonationDate.	Items received; increases stock.
Need	NeedID, FamilyID, ItemID, Quantity, Status. 1:N with Distribution.	Family's requirement; one per family and item.
Distribution	DistributionID, NeedID, StockID, VolunteerID, Quantity, DistDate.	Core transaction linking need, stock and volunteer.
Assignment	AssignmentID, VolunteerID, CampID, Role, StartDate, EndDate.	Volunteer duty at a camp (resolves M:N).
Referral	ReferralID, PersonID, CampID, Facility, Reason, ReferralDate, Status.	Medical referral from camp to a facility.

CHAPTER 2

LOGICAL DATABASE DESIGN

This chapter shows how the approved ER model was turned into a relational design, tested against the normal forms and confirmed with a working SQLite prototype.

2.1 Relational Schema

In the relational model, data is held in relations that are tied together by keys [2]. Following the usual ER-to-relational mapping, every entity became a table and every one-to-many relationship became a foreign key on the “many” side, which yields the fourteen relations in Table 5. The schema listing after the table marks primary keys as [PK] and foreign keys as [FK].

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	Disaster	Stores each disaster event and its severity.
2	Location	Stores affected places linked to a disaster.
3	Camp	Stores relief camps, capacity and status.
4	Warehouse	Stores warehouses and their managers.
5	Family	Stores registered families and their camp.
6	Person	Stores individual members and medical conditions.
7	Volunteer	Stores volunteer details and availability.
8	ReliefItem	Catalogue of relief items with category and unit.
9	Stock	Stores quantity of each item in each warehouse.
10	Donation	Stores donations received by warehouses.
11	Need	Stores the quantity of each item a family requires.
12	Distribution	Records each handover from stock against a need.
13	Assignment	Records volunteer duties at camps.
14	Referral	Records medical referrals of persons.

Relational schema (PK = primary key, FK = foreign key):

```
Disaster(DisasterID [PK], DisasterName, DisasterType, StartDate, Severity, Status)
Location(LocationID [PK], Village, District, State, DisasterID [FK])
Camp(CampID [PK], CampName, LocationID [FK], Capacity, Status)
Warehouse(WarehouseID [PK], WarehouseName, LocationID [FK], ManagerName)
Family(FamilyID [PK], HeadOfFamily, ContactInfo, CampID [FK], RegisteredOn)
Person(PersonID [PK], FamilyID [FK], FullName, Age, Gender, MedicalCondition)
Volunteer(VolunteerID [PK], FullName, Phone, Skill, Status)
ReliefItem(ItemID [PK], ItemName, Category, Unit)
Stock(StockID [PK], ItemID [FK], WarehouseID [FK], Quantity, LastUpdated)
Donation(DonationID [PK], DonorName, ItemID [FK], WarehouseID [FK], Quantity,
DonationDate)
Need(NeedID [PK], FamilyID [FK], ItemID [FK], Quantity, Status)
Distribution(DistributionID [PK], NeedID [FK], StockID [FK], VolunteerID [FK],
Quantity, DistDate)
Assignment(AssignmentID [PK], VolunteerID [FK], CampID [FK], Role, StartDate,
EndDate)
Referral(ReferralID [PK], PersonID [FK], CampID [FK], Facility, Reason,
ReferralDate, Status)
```

2.2 Keys and Integrity Constraints

Table 6 gives the keys and integrity constraints, together with the triggers that handle business rules that constraints alone cannot express.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
Disaster	Primary Key	DisasterID	Uniquely identifies each disaster.
	Unique / Check	DisasterName; DisasterType, Severity, Status	One record per event; only permitted type, severity and status values.
Location	Primary / Foreign Key	LocationID; DisasterID refers to Disaster	Every location belongs to an existing disaster (ON DELETE RESTRICT).
	Unique / Default	(Village, District, DisasterID); State = 'Telangana'	A place is not registered twice for one disaster.
Camp	Primary / Foreign Key	CampID; LocationID refers to Location	A camp must be set up at a valid location.
	Unique / Check	CampName; Capacity > 0; Status IN ('Open','Full','Closed')	Unique camp names, positive capacity, valid status (default 'Open').

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
Warehouse	Primary / Foreign Key	WarehouseID; LocationID refers to Location	A warehouse must be at a valid location.
	Unique / Not Null	WarehouseName; ManagerName	Unique name and a responsible manager.
Family	Primary / Foreign Key	FamilyID; CampID refers to Camp	A family belongs to an existing camp; the camp cannot be deleted while families exist (RESTRICT).
	Not Null / Default	HeadOfFamily; RegisteredOn = CURRENT_DATE	Head of family is mandatory; registration date defaults to today.
Person	Primary / Foreign Key	PersonID; FamilyID refers to Family	Members are removed with their family (ON DELETE CASCADE).
	Check	Age BETWEEN 0 AND 120; Gender IN ('F','M','O')	Rejects impossible ages and invalid gender codes.
Volunteer	Primary Key / Unique	VolunteerID; Phone	One record per volunteer phone number.
	Check / Default	Status IN ('Available','Assigned','Inactive')	Valid availability status (default 'Available').
ReliefItem	Primary Key / Unique	ItemID; ItemName	One catalogue row per item.
	Check	Category IN ('Food','Water','Shelter','Clothing','Medical')	Valid item categories.
Stock	Primary / Foreign Key	StockID; ItemID refers to ReliefItem; WarehouseID refers to Warehouse	Stock refers to valid items and warehouses.
	Unique	(ItemID, WarehouseID)	Only one stock row per item and warehouse.
	Check / Default	Quantity >= 0 (default 0); LastUpdated = CURRENT_TIMESTAMP	Stock can never become negative.
Donation	Primary / Foreign Key	DonationID; ItemID, WarehouseID	Donations refer to valid items and warehouses.
	Check / Trigger	Quantity > 0; trg_donation_stock	Donated quantity is added to Stock automatically (insert or update).
Need	Primary / Foreign Key	NeedID; FamilyID refers to Family; ItemID refers to ReliefItem	Needs refer to valid families and items.
	Unique	(FamilyID, ItemID)	Prevents a duplicate need, and so a duplicate entitlement.
	Check / Default	Quantity > 0; Status IN ('Pending','Partial','Fulfilled')	Valid quantity and status (default 'Pending').
Distribution	Primary / Foreign Key	DistributionID; NeedID, StockID, VolunteerID	Every handover is linked to a need, a stock row and a volunteer.
	Check / Trigger	Quantity > 0; trg_dist_validate	Rejects over-allocation, duplicate entitlement, item mismatch and insufficient stock.
	Trigger	trg_dist_apply	Reduces stock and updates the need status after a valid distribution.
Assignment	Primary / Foreign Key	AssignmentID; VolunteerID, CampID	Assignments refer to valid volunteers and camps.
	Check	EndDate IS NULL OR EndDate >= StartDate	A duty cannot end before it starts.
Referral	Primary / Foreign Key	ReferralID; PersonID, CampID	Referrals refer to valid persons and camps.
	Check / Default	Status IN ('Open','Attended','Closed')	Valid status (default 'Open').

Referential actions work as follows: ON DELETE RESTRICT stops the removal of records that other rows still depend on (camps, items, warehouses, needs), whereas ON DELETE CASCADE deletes a family's members and needs along with the family. Every connection runs PRAGMA foreign_keys = ON so that these checks are active.

2.3 Functional Dependencies

The main functional dependencies (FDs) of the design appear below, and in each relation every determinant is a candidate key. No dependency from District to State is assumed because a district name can exist in several states.

- DisasterID → DisasterName, DisasterType, StartDate, Severity, Status; DisasterName → DisasterID (candidate key).
- LocationID → Village, District, State, DisasterID.
- CampID → CampName, LocationID, Capacity, Status; CampName → CampID.
- WarehouseID → WarehouseName, LocationID, ManagerName; WarehouseName → WarehouseID.
- FamilyID → HeadOfFamily, ContactInfo, CampID, RegisteredOn.
- PersonID → FamilyID, FullName, Age, Gender, MedicalCondition.
- VolunteerID → FullName, Phone, Skill, Status; Phone → VolunteerID.
- ItemID → ItemName, Category, Unit; ItemName → ItemID.
- StockID → ItemID, WarehouseID, Quantity, LastUpdated; (ItemID, WarehouseID) → StockID, Quantity, LastUpdated.
- DonationID → DonorName, ItemID, WarehouseID, Quantity, DonationDate.
- NeedID → FamilyID, ItemID, Quantity, Status; (FamilyID, ItemID) → NeedID, Quantity, Status.
- DistributionID → NeedID, StockID, VolunteerID, Quantity, DistDate.
- AssignmentID → VolunteerID, CampID, Role, StartDate, EndDate.
- ReferralID → PersonID, CampID, Facility, Reason, ReferralDate, Status.

2.4 Normalization up to Third Normal Form (3NF)

The paper registers from Section 1.1 amount to one wide unnormalized list that repeats family, camp, location, item and volunteer details on each line. For first normal form (1NF), repeating groups were taken out: members went to Person and needs to Need, so each column holds one value. For second normal form (2NF), attributes were split according to which part of the key determines them: item descriptions went to ReliefItem, and stock quantity depends on the complete key (ItemID, WarehouseID) [3], [4].

For third normal form (3NF), transitive dependencies were eliminated: place details are kept once in Location instead of in Camp and Warehouse, and Location points to Disaster. Since every determinant in Section 2.3 is a candidate key, the relations also meet Boyce-Codd normal form (BCNF). Table 7 gives the outcome per relation.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
Disaster	Yes	Yes	Yes	Yes	Single-valued attributes; key is a single column.
Location	Yes	Yes	Yes	Yes	Separated from Camp and Warehouse so place details are stored once. No FD between District and State is assumed, because district names repeat across states.
Camp	Yes	Yes	Yes	Yes	Camp details depend only on CampID; place details live in Location.
Warehouse	Yes	Yes	Yes	Yes	Same pattern as Camp.
Family	Yes	Yes	Yes	Yes	Members moved to Person to remove a repeating group (1NF).
Person	Yes	Yes	Yes	Yes	One row per member; no multi-valued columns.
Volunteer	Yes	Yes	Yes	Yes	Duties moved to Assignment so one volunteer can serve many camps.
ReliefItem	Yes	Yes	Yes	Yes	Item description stored once, not repeated in Stock or Need.
Stock	Yes	Yes	Yes	Yes	Quantity depends on the whole (ItemID, WarehouseID), so no partial dependency (2NF). Quantity is a balance maintained by triggers.
Donation	Yes	Yes	Yes	Yes	Each donation is one row; donor name kept as a descriptive attribute.
Need	Yes	Yes	Yes	Yes	Status is maintained by trigger from the sum of Distribution rows (controlled derived attribute kept for fast queries).
Distribution	Yes	Yes	Yes	Yes	Resolves the many-to-many link between Need and Stock and adds the volunteer.
Assignment	Yes	Yes	Yes	Yes	Resolves the many-to-many link between Volunteer and Camp.
Referral	Yes	Yes	Yes	Yes	Refers to a person and a camp; no derived data stored.

2.5 Data Dictionary

Table 8 describes all attributes of the fourteen tables. Data types are SQLite's, and the descriptions and constraints were produced from the implemented schema so the dictionary stays consistent with the database.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
Disaster	DisasterID	INTEGER	PK	Unique identifier of the disaster record
Disaster	DisasterName	TEXT	NOT NULL; UNIQUE	Official name of the disaster event
Disaster	DisasterType	TEXT	NOT NULL; CHECK (DisasterType IN ('Flood','Cyclone','Earthquake','Drought','Fire'))	Category of disaster
Disaster	StartDate	DATE	NOT NULL	Date the disaster or duty began
Disaster	Severity	TEXT	NOT NULL; CHECK (Severity IN ('Low','Medium','High','Critical'))	Assessed severity level
Disaster	Status	TEXT	NOT NULL; CHECK (Status IN ('Active','Closed')); DEFAULT 'Active'	Active or Closed

Table	Attribute	Data Type	Key / Constraint	Description
Location	LocationID	INTEGER	PK	Unique identifier of the location record
Location	Village	TEXT	NOT NULL; UNIQUE (Village, District, DisasterID)	Village or town affected
Location	District	TEXT	NOT NULL; UNIQUE (Village, District, DisasterID)	District of the location
Location	State	TEXT	NOT NULL; DEFAULT 'Telangana'	State of the location
Location	DisasterID	INTEGER	FK → Disaster; NOT NULL; UNIQUE (Village, District, DisasterID)	Disaster that affected the place
Camp	CampID	INTEGER	PK	Unique identifier of the camp record
Camp	CampName	TEXT	NOT NULL; UNIQUE	Name of the relief camp
Camp	LocationID	INTEGER	FK → Location; NOT NULL	Location of the camp
Camp	Capacity	INTEGER	NOT NULL; CHECK (Capacity > 0)	Maximum number of people the camp can shelter
Camp	Status	TEXT	NOT NULL; CHECK (Status IN ('Open','Full','Closed')); DEFAULT 'Open'	Open, Full or Closed
Warehouse	WarehouseID	INTEGER	PK	Unique identifier of the warehouse record
Warehouse	WarehouseName	TEXT	NOT NULL; UNIQUE	Name of the warehouse
Warehouse	LocationID	INTEGER	FK → Location; NOT NULL	Location of the warehouse
Warehouse	ManagerName	TEXT	NOT NULL	Person responsible for the warehouse
Family	FamilyID	INTEGER	PK	Unique identifier of the family record
Family	HeadOfFamily	TEXT	NOT NULL	Name of the head of the family
Family	ContactInfo	TEXT		Phone number or other contact detail
Family	CampID	INTEGER	FK → Camp; NOT NULL	Camp in which the family is registered
Family	RegisteredOn	DATE	NOT NULL; DEFAULT CURRENT_DATE	Date of registration in the camp
Person	PersonID	INTEGER	PK	Unique identifier of the person record
Person	FamilyID	INTEGER	FK → Family; NOT NULL	Family to which the person belongs
Person	FullName	TEXT	NOT NULL	Full name of the person or volunteer
Person	Age	INTEGER	NOT NULL; CHECK (Age BETWEEN 0 AND 120)	Age in years
Person	Gender	TEXT	NOT NULL; CHECK (Gender IN ('F','M','O'))	F, M or O
Person	MedicalCondition	TEXT	DEFAULT 'None'	Known medical condition (default None)
Volunteer	VolunteerID	INTEGER	PK	Unique identifier of the volunteer record
Volunteer	FullName	TEXT	NOT NULL	Full name of the person or volunteer
Volunteer	Phone	TEXT	NOT NULL; UNIQUE	Unique contact number
Volunteer	Skill	TEXT	NOT NULL	Main skill or function of the volunteer
Volunteer	Status	TEXT	NOT NULL; CHECK (Status IN ('Available','Assigned','Inactive')); DEFAULT 'Available'	Available, Assigned or Inactive
ReliefItem	ItemID	INTEGER	PK	Unique identifier of the reliefitem record
ReliefItem	ItemName	TEXT	NOT NULL; UNIQUE	Name of the relief item
ReliefItem	Category	TEXT	NOT NULL; CHECK (Category IN ('Food','Water','Shelter','Clothing','Medical'))	Item category
ReliefItem	Unit	TEXT	NOT NULL	Unit of measure
Stock	StockID	INTEGER	PK	Unique identifier of the stock record
Stock	ItemID	INTEGER	FK → ReliefItem; NOT NULL; UNIQUE (ItemID, WarehouseID)	Item held

Table	Attribute	Data Type	Key / Constraint	Description
Stock	WarehouseID	INTEGER	FK → Warehouse; NOT NULL; UNIQUE (ItemID, WarehouseID)	Warehouse holding the stock
Stock	Quantity	INTEGER	NOT NULL; CHECK (Quantity >= 0); DEFAULT 0	Units currently available
Stock	LastUpdated	TIMESTAMP	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp of the last change
Donation	DonationID	INTEGER	PK	Unique identifier of the donation record
Donation	DonorName	TEXT	NOT NULL	Name of the donor or organisation
Donation	ItemID	INTEGER	FK → ReliefItem; NOT NULL	Item donated
Donation	WarehouseID	INTEGER	FK → Warehouse; NOT NULL	Warehouse that receives the donation
Donation	Quantity	INTEGER	NOT NULL; CHECK (Quantity > 0)	Units donated
Donation	DonationDate	DATE	NOT NULL; DEFAULT CURRENT_DATE	Date the donation was received
Need	NeedID	INTEGER	PK	Unique identifier of the need record
Need	FamilyID	INTEGER	FK → Family; NOT NULL; UNIQUE (FamilyID, ItemID)	Family that has the need
Need	ItemID	INTEGER	FK → ReliefItem; NOT NULL; UNIQUE (FamilyID, ItemID)	Item required
Need	Quantity	INTEGER	NOT NULL; CHECK (Quantity > 0)	Total units required by the family
Need	Status	TEXT	NOT NULL; CHECK (Status IN ('Pending','Partial','Fulfilled')); DEFAULT 'Pending'	Pending, Partial or Fulfilled
Distribution	DistributionID	INTEGER	PK	Unique identifier of the distribution record
Distribution	NeedID	INTEGER	FK → Need; NOT NULL	Need being served
Distribution	StockID	INTEGER	FK → Stock; NOT NULL	Stock row supplying the items
Distribution	VolunteerID	INTEGER	FK → Volunteer; NOT NULL	Volunteer who handed over the items
Distribution	Quantity	INTEGER	NOT NULL; CHECK (Quantity > 0)	Units handed over
Distribution	DistDate	DATE	NOT NULL; DEFAULT CURRENT_DATE	Date of the distribution
Assignment	AssignmentID	INTEGER	PK	Unique identifier of the assignment record
Assignment	VolunteerID	INTEGER	FK → Volunteer; NOT NULL	Volunteer assigned
Assignment	CampID	INTEGER	FK → Camp; NOT NULL	Camp where the volunteer serves
Assignment	Role	TEXT	NOT NULL	Duty performed at the camp
Assignment	StartDate	DATE	NOT NULL	Date the disaster or duty began
Assignment	EndDate	DATE	CHECK EndDate IS NULL OR EndDate >= StartDate	End of duty; NULL while active
Referral	ReferralID	INTEGER	PK	Unique identifier of the referral record
Referral	PersonID	INTEGER	FK → Person; NOT NULL	Person being referred
Referral	CampID	INTEGER	FK → Camp; NOT NULL	Camp from which the referral is made
Referral	Facility	TEXT	NOT NULL	Hospital or health centre referred to
Referral	Reason	TEXT	NOT NULL	Reason for the referral
Referral	ReferralDate	DATE	NOT NULL; DEFAULT CURRENT_DATE	Date of the referral
Referral	Status	TEXT	NOT NULL; CHECK (Status IN ('Open','Attended','Closed')); DEFAULT 'Open'	Open, Attended or Closed

2.6 Implementation of Logical Design: DDL and Sample Data

The schema was created in SQLite [5] with the DDL given in Appendix A, and Table 9 summarises the setup. CREATE TABLE and CREATE TRIGGER examples follow, along with sample records read back from the database.

Table 9. Database Implementation Environment

Item	Details
DBMS	SQLite
DBMS Version	3.45.1 (accessed through Python 3.12)
Number of Tables	14 tables, 3 triggers and 3 indexes
Sample Data	94 illustrative records, e.g. Family 6, Person 27, Need 12, Stock 8, Distribution 5

Representative DDL:

```
CREATE TABLE Family (
  FamilyID     INTEGER PRIMARY KEY AUTOINCREMENT,
  HeadOfFamily TEXT NOT NULL,
  ContactInfo  TEXT,
  CampID       INTEGER NOT NULL
               REFERENCES Camp(CampID) ON DELETE RESTRICT,
  RegisteredOn DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE Need (
  NeedID   INTEGER PRIMARY KEY AUTOINCREMENT,
  FamilyID INTEGER NOT NULL
           REFERENCES Family(FamilyID) ON DELETE CASCADE,
  ItemID   INTEGER NOT NULL
           REFERENCES ReliefItem(ItemID) ON DELETE RESTRICT,
  Quantity INTEGER NOT NULL CHECK (Quantity > 0),
  Status   TEXT NOT NULL DEFAULT 'Pending'
           CHECK (Status IN ('Pending','Partial','Fulfilled')),
  UNIQUE (FamilyID, ItemID)
);

CREATE TABLE Distribution (
  DistributionID INTEGER PRIMARY KEY AUTOINCREMENT,
  NeedID      INTEGER NOT NULL REFERENCES Need(NeedID),
  StockID     INTEGER NOT NULL REFERENCES Stock(StockID),
  VolunteerID INTEGER NOT NULL REFERENCES Volunteer(VolunteerID),
  Quantity    INTEGER NOT NULL CHECK (Quantity > 0),
  DistDate    DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TRIGGER trg_dist_validate BEFORE INSERT ON Distribution
BEGIN
  SELECT CASE WHEN (SELECT Status FROM Need
                    WHERE NeedID = NEW.NeedID) = 'Fulfilled'
    THEN RAISE(ABORT, 'Duplicate entitlement: need already fulfilled')
  END;
  -- further checks: item mismatch, over-allocation,
  -- insufficient stock (see Appendix A)
END;

CREATE TRIGGER trg_dist_apply AFTER INSERT ON Distribution
BEGIN
  UPDATE Stock SET Quantity = Quantity - NEW.Quantity,
         LastUpdated = CURRENT_TIMESTAMP
   WHERE StockID = NEW.StockID;
  UPDATE Need SET Status = CASE
     WHEN (SELECT SUM(Quantity) FROM Distribution
           WHERE NeedID = NEW.NeedID) >= Quantity
     THEN 'Fulfilled' ELSE 'Partial' END
   WHERE NeedID = NEW.NeedID;
END;
```

Representative sample data (SELECT queries on the loaded database):

```
SELECT NeedID, FamilyID, ItemID, Quantity, Status FROM Need;
+--------+----------+--------+----------+-----------+
| NeedID | FamilyID | ItemID | Quantity | Status    |
+--------+----------+--------+----------+-----------+
| 1      | 1        | 1      | 50       | Fulfilled |
| 2      | 1        | 3      | 5        | Partial   |
| 3      | 2        | 2      | 80       | Pending   |
| 4      | 3        | 4      | 1        | Pending   |
| 5      | 3        | 1      | 60       | Partial   |
| 6      | 4        | 5      | 1        | Pending   |
| 7      | 5        | 2      | 100      | Fulfilled |
| 8      | 5        | 3      | 7        | Pending   |
| 9      | 6        | 1      | 20       | Fulfilled |
| 10     | 2        | 3      | 4        | Pending   |
| 11     | 4        | 6      | 5        | Pending   |
| 12     | 6        | 5      | 1        | Pending   |
+--------+----------+--------+----------+-----------+
SELECT StockID, ItemID, WarehouseID, Quantity FROM Stock;
+---------+--------+-------------+----------+
| StockID | ItemID | WarehouseID | Quantity |
+---------+--------+-------------+----------+
| 1       | 1      | 1           | 430      |
| 2       | 2      | 1           | 1100     |
| 3       | 3      | 1           | 98       |
| 4       | 4      | 2           | 22       |
| 5       | 5      | 2           | 28       |
| 6       | 1      | 2           | 130      |
| 7       | 3      | 2           | 9        |
| 8       | 6      | 1           | 60       |
+---------+--------+-------------+----------+
```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

This chapter explains how the Python application makes use of the approved logical design, reading from and writing to the SQLite database for every operation.

3.1 Application Overview

The application, relief_app.py, is a Python program that talks to the SQLite database through the standard sqlite3 module. Camp coordinators, warehouse managers and volunteers use it to register families and members, log needs and donations, hand out items, post volunteers and record referrals, while administrators run six reports. Every piece of data comes from the 14-table database and nothing is hard-coded. The database itself enforces the business rules through constraints and triggers, and the program turns any rejection into a readable message.

3.2 Development Environment and Database Connectivity

Table 10 gives the development environment. All writes are parameterised queries inside transactions, as explained in Section 3.5.

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.12
DBMS	SQLite 3.45.1
IDE / Framework / GUI Toolkit	Text editor and terminal; console application using only the Python standard library
Database Connector / Driver	sqlite3 module (DB-API 2.0) [6]
Operating Environment	Linux (Ubuntu 24); portable to Windows and macOS
Repository Link	[Enter GitHub repository link]

3.3 Application Modules and Database Mapping

Table 11 links each application module to the tables it works with.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Camp and Family Registration	Register families and members	Camp, Family, Person	INSERT, UPDATE, SELECT
Needs Management	Record and track family needs	Need, ReliefItem, Family	INSERT, SELECT
Stock and Donations	Maintain stock; record donations	Stock, Donation, Warehouse, ReliefItem	INSERT, SELECT (stock updated by trigger)
Distribution	Hand over items against a need	Distribution, Need, Stock, Volunteer	INSERT (validated by triggers)
Volunteer Assignment	Assign volunteers to camps	Volunteer, Assignment, Camp	INSERT, SELECT
Referrals	Record medical referrals	Referral, Person, Camp	INSERT, SELECT
Reports	Operational reports	All relevant tables	SELECT with JOIN, GROUP BY, subquery, NOT EXISTS

3.4 CRUD Operations

Table 12 shows how each CRUD operation is supported and where the evidence can be found.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Register family, add person, add need, record donation, record distribution	Family, Person, Need, Donation, Distribution, Stock	Figure 2 and Figure 3
Read / Search	Camp, need, stock, audit and volunteer reports	Camp, Family, Need, ReliefItem, Stock, Distribution, Volunteer	Figure 4 and Figure 5
Update	Change family contact; stock and need status updated by distribution and donation	Family, Stock, Need	Figure 2 (update_contact) and Figure 3

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Delete / Controlled Removal	Delete camp is refused while families refer to it	Camp, Family	Figure 2 (delete_camp rejected)

3.5 Business Rules, Validation and Error Handling

Table 13 shows how the application guards against invalid input and rule violations. The rules live in the database, so they apply to every user and program. Queries are always parameterised, which keeps user input out of the SQL text, and each write sits in a transaction that commits on success and rolls back on failure, keeping stock and need status consistent (ACID).

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
No duplicate need for a family and item	UNIQUE (FamilyID, ItemID) on Need; the application catches IntegrityError.	REJECTED: UNIQUE constraint failed: Need.FamilyID, Need.ItemID
No duplicate entitlement	Trigger trg_dist_validate blocks a distribution for a Fulfilled need.	REJECTED: Duplicate entitlement: need already fulfilled
No over-allocation	Trigger compares the quantity with need minus quantity already distributed.	REJECTED: Over-allocation: quantity exceeds remaining need
Stock availability	Trigger compares the quantity with the stock row; CHECK keeps Quantity >= 0.	REJECTED: Insufficient stock in selected warehouse
Item must match	Trigger compares the item of the need with the item of the stock row.	REJECTED: Item mismatch: stock item differs from the needed item
Valid references and values	FOREIGN KEY and CHECK constraints with PRAGMA foreign_keys = ON.	REJECTED: FOREIGN KEY constraint failed
Safe SQL and atomic updates	Parameterised queries (?) only; each write runs in a transaction (with con:) that commits on success and rolls back on error.	No partial updates after a rejected request

3.6 Reports and Database Operations

The SQL queries in Table 14 are run from the application, and Figure 4 and Figure 5 show what they output.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Camp occupancy	LEFT JOIN, COUNT, GROUP BY	Figure 4: people sheltered and percentage of capacity per camp
Unmet needs by camp	Multi-table JOIN, SUM, GROUP BY	Figure 4: remaining quantity per family and item
Stock alerts	JOIN, SUM, CASE	Figure 4: total stock per item with LOW flag
Distribution audit trail	Six-table JOIN	Figure 5: date, family, item, quantity, warehouse, volunteer
Volunteer workload	Correlated subqueries, GROUP_CONCAT	Figure 5: distributions handled and active camp
Families without aid	NOT EXISTS subquery	Figure 5: families with no distribution

3.7 Application Screens

As the application runs in the console, the screens below are its real console output. Figure 2 covers data entry, updates and constraint checks, Figure 3 covers distribution transactions and rule validation, and Figures 4 and 5 present the reports.

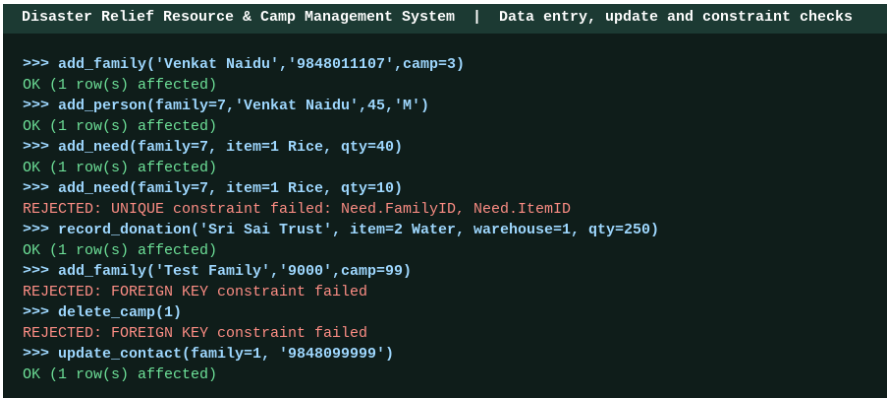


Figure 2. Family registration, need entry, donation and constraint checks

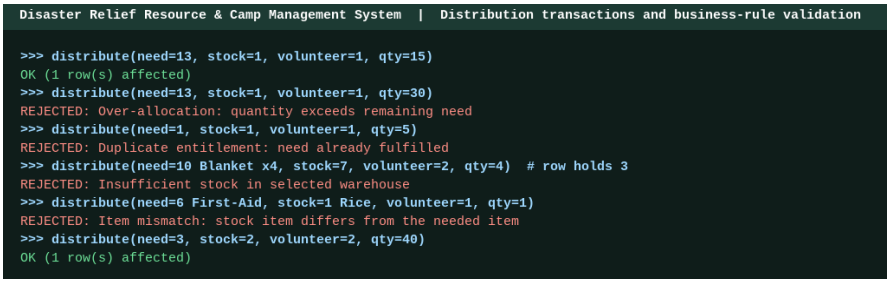


Figure 3. Distribution transactions with business-rule validation

```
Disaster Relief Resource & Camp Management System  |  Report: camp occupancy, unmet needs, stock alerts

Camp occupancy (JOIN + GROUP BY)
+------------------------+----------+-----------+---------+
| CampName               | Capacity | Sheltered | PctFull |
+------------------------+----------+-----------+---------+
| Siddipet Relief Camp A | 300      | 9         | 3.0     |
| Husnabad Camp B        | 180      | 9         | 5.0     |
| Gajwel Camp C          | 220      | 11        | 5.0     |
+------------------------+----------+-----------+---------+

Unmet needs by camp (multi-table JOIN)
+------------------------+--------------+----------------+-----------+---------+
| CampName               | Family       | Item           | Remaining | Status  |
+------------------------+--------------+----------------+-----------+---------+
| Siddipet Relief Camp A | Ravi Kumar   | Blanket        | 3         | Partial |
| Siddipet Relief Camp A | Lakshmi Devi | Drinking Water | 40        | Partial |
| Siddipet Relief Camp A | Lakshmi Devi | Blanket        | 4         | Pending |
| Husnabad Camp B        | Anwar Khan   | Tarpaulin Tent | 1         | Pending |
| Husnabad Camp B        | Anwar Khan   | Rice           | 40        | Partial |
| Husnabad Camp B        | Sunita Reddy | First-Aid Kit  | 1         | Pending |
| Husnabad Camp B        | Sunita Reddy | Cooking Oil    | 5         | Pending |
| Gajwel Camp C          | Mahesh Goud  | Blanket        | 7         | Pending |
| Gajwel Camp C          | Padma Rao    | First-Aid Kit  | 1         | Pending |
| Gajwel Camp C          | Venkat Naidu | Rice           | 25        | Partial |
+------------------------+--------------+----------------+-----------+---------+

Total stock per item with low-stock flag (aggregate + CASE)
+----------------+-------+----------+-------+
| ItemName       | Unit  | TotalQty | Level |
+----------------+-------+----------+-------+
| Tarpaulin Tent | piece | 22       | LOW   |
| First-Aid Kit  | kit   | 28       | LOW   |
| Cooking Oil    | litre | 60       | OK    |
| Blanket        | piece | 101      | OK    |
| Rice           | kg    | 545      | OK    |
| Drinking Water | litre | 1310     | OK    |
+----------------+-------+----------+-------+
```

Figure 4. Report: camp occupancy, unmet needs and stock alerts

```
Disaster Relief Resource & Camp Management System  |  Report: audit trail, volunteer workload, families without aid

Distribution audit trail (traceability)
+----+------------+--------------+----------------+-----+---------------------------+--------------+
| ID | Date       | Family       | Item           | Qty | FromWarehouse             | Volunteer    |
+----+------------+--------------+----------------+-----+---------------------------+--------------+
| 1  | 2026-10-09 | Ravi Kumar   | Rice           | 50  | Siddipet Central Warehouse | Anitha Rao   |
| 2  | 2026-10-10 | Ravi Kumar   | Blanket        | 2   | Siddipet Central Warehouse | Anitha Rao   |
| 3  | 2026-10-10 | Anwar Khan   | Rice           | 20  | Karimnagar Depot           | Imran Shaikh |
| 4  | 2026-10-11 | Mahesh Goud  | Drinking Water | 100 | Siddipet Central Warehouse | Imran Shaikh |
| 5  | 2026-10-11 | Padma Rao    | Rice           | 20  | Siddipet Central Warehouse | Anitha Rao   |
| 6  | 2026-10-12 | Venkat Naidu | Rice           | 15  | Siddipet Central Warehouse | Anitha Rao   |
| 7  | 2026-10-12 | Lakshmi Devi | Drinking Water | 40  | Siddipet Central Warehouse | Imran Shaikh |
+----+------------+--------------+----------------+-----+---------------------------+--------------+

Volunteer workload (subquery)
+---------------+-------------+---------------+-----------------------+
| FullName      | Skill       | Distributions | ActiveCamp            |
+---------------+-------------+---------------+-----------------------+
| Anitha Rao    | Logistics   | 4             | Siddipet Relief Camp A |
| Imran Shaikh  | Driver      | 3             | Siddipet Relief Camp A |
| Suresh Babu   | Medical Aid | 0             | Husnabad Camp B        |
| Divya Nair    | Registration| 0             | Gajwel Camp C          |
| Karthik Yadav | Cooking     | 0             | None                   |
+---------------+-------------+---------------+-----------------------+

Families that have received no aid (NOT EXISTS)
+----------+--------------+-----------------+
| FamilyID | HeadOfFamily | CampName        |
+----------+--------------+-----------------+
| 4        | Sunita Reddy | Husnabad Camp B |
+----------+--------------+-----------------+
```

Figure 5. Report: audit trail, volunteer workload and families without aid

3.8 Testing and Results

Table 15 gives the positive and negative test cases. Each one ran against a copy of the database through run_tests.py, and the actual results are the values the program recorded.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Register family	Family 'Venkat Naidu' to Camp 3 with 2 members	Rows inserted in Family and Person	Family ID 7 created; 2 persons stored	Pass
TC-02	Register need	Family 7, Rice, qty 40	Need created with status Pending	NeedID 13, status Pending	Pass
TC-03	One need per family/item	Second Rice need for family 7	Rejected by UNIQUE(FamilyID, ItemID)	REJECTED: UNIQUE constraint failed: Need.FamilyID, Need.ItemID	Pass
TC-04	Valid distribution	Need 13, 15 kg Rice from Stock 1	Stock falls by 15; need becomes Partial	Stock 430 -> 415; status Partial	Pass
TC-05	Over-allocation	Need 13 has 25 kg left; try 30 kg	Rejected; stock unchanged	REJECTED: Over-allocation: quantity exceeds remaining need; stock still 415	Pass
TC-06	Duplicate entitlement	Need 1 (Ravi Kumar, Rice) is already Fulfilled	Rejected as duplicate	REJECTED: Duplicate entitlement: need already fulfilled	Pass
TC-07	Insufficient stock	Need 4 blankets from a warehouse row holding 3	Rejected; need and stock unchanged	REJECTED: Insufficient stock in selected warehouse; need status Pending, stock 3	Pass
TC-08	Item mismatch	Need for First-Aid Kit served from Rice stock	Rejected	REJECTED: Item mismatch: stock item differs from the needed item	Pass
TC-09	Donation updates stock	Donate 250 L water to Warehouse 1	Stock row increases by 250	Stock 1100 -> 1350	Pass
TC-10	Foreign key check	Family linked to non-existent Camp 99	Rejected (FOREIGN KEY)	REJECTED: FOREIGN KEY constraint failed	Pass
TC-11	Controlled removal	Delete Camp 1 that still has families	Rejected; Camp 1 remains	REJECTED: FOREIGN KEY constraint failed	Pass
TC-12	Update record	Change contact of Family 1	ContactInfo updated	9848099999	Pass
TC-13	CHECK constraint	Person with Age = 150	Rejected by CHECK (Age 0-120)	REJECTED: CHECK constraint failed: Age BETWEEN 0 AND 120	Pass

All 13 test cases passed (13 Pass, 0 Fail), as recorded in Table 15.

The results confirm that the application works with the designed database and applies its main business rules. Duplicate needs, repeat entitlement, over-allocation, shortage of stock, item mismatch, broken references, restricted deletion and invalid values were all refused with specific messages, leaving stock and need status untouched, while valid donations and distributions changed stock and status as expected.

CHAPTER 4
CONCLUSION

This project tackled the coordination failures of manual disaster relief by designing and building a relational database for camps, families, needs, stock, donations, volunteers and

distributions. The fourteen-entity ER model was mapped to third normal form tables in SQLite, with keys, check, unique and default constraints and three triggers that apply the main business rules within the database.

A Python application using the standard sqlite3 module builds on this design for registration, donations, distributions, postings and referrals, and for six analytical reports. All thirteen test cases passed: duplicate needs, repeat entitlement, over-allocation, insufficient stock, item mismatch and invalid references were refused, and valid operations updated stock and need status correctly. The project showed that constraints, triggers and normalization together keep shared data consistent and auditable.

4.1 Limitations and Future Enhancements

- The program runs in the console; a graphical or web front end on the same database would be easier for camp staff.
- SQLite supports one user at a time; PostgreSQL or MySQL would be needed for concurrent use across camps.
- Because triggers maintain need status and stock quantity, any bulk data load must go through the same tables and rules.
- There is no user authentication, role-based access or audit of data edits.
- The low-stock threshold is fixed inside the report query; a reorder level per item could be stored in ReliefItem.
- The browser prototype in Appendix B holds data in memory and is not linked to the database.

REFERENCES

[1] Sphere Association, The Sphere Handbook: Humanitarian Charter and Minimum Standards in Humanitarian Response, 4th ed. Geneva, Switzerland: Sphere Association, 2018.

[2] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] SQLite Consortium, "SQLite documentation," SQLite. Accessed: Oct. 12, 2026. [Online]. Available: https://www.sqlite.org/docs.html

[6] Python Software Foundation, "sqlite3 — DB-API 2.0 interface for SQLite databases," Python 3 Documentation. Accessed: Oct. 12, 2026. [Online]. Available: https://docs.python.org/3/library/sqlite3.html

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The full database script is split into schema.sql (data definition language (DDL): tables, constraints, triggers, indexes) and data.sql (data manipulation language (DML): sample data). Both files are printed below and supplied with the source code.

```
-- Disaster Relief Resource & Camp Management System : schema (SQLite 3)
PRAGMA foreign_keys = ON;
CREATE TABLE Disaster (
    DisasterID   INTEGER PRIMARY KEY AUTOINCREMENT,
    DisasterName TEXT NOT NULL UNIQUE,
    DisasterType TEXT NOT NULL CHECK (DisasterType IN ('Flood','Cyclone','Earthquake','Drought','Fire')),
    StartDate    DATE NOT NULL,
    Severity     TEXT NOT NULL CHECK (Severity IN ('Low','Medium','High','Critical')),
    Status       TEXT NOT NULL DEFAULT 'Active' CHECK (Status IN ('Active','Closed'))
);
CREATE TABLE Location (
    LocationID INTEGER PRIMARY KEY AUTOINCREMENT,
    Village    TEXT NOT NULL,
    District   TEXT NOT NULL,
    State      TEXT NOT NULL DEFAULT 'Telangana',
    DisasterID INTEGER NOT NULL REFERENCES Disaster(DisasterID) ON DELETE RESTRICT,
    UNIQUE (Village, District, DisasterID)
);
CREATE TABLE Camp (
    CampID     INTEGER PRIMARY KEY AUTOINCREMENT,
    CampName   TEXT NOT NULL UNIQUE,
    LocationID INTEGER NOT NULL REFERENCES Location(LocationID) ON DELETE RESTRICT,
    Capacity   INTEGER NOT NULL CHECK (Capacity > 0),
    Status     TEXT NOT NULL DEFAULT 'Open' CHECK (Status IN ('Open','Full','Closed'))
);
CREATE TABLE Warehouse (
    WarehouseID   INTEGER PRIMARY KEY AUTOINCREMENT,
    WarehouseName TEXT NOT NULL UNIQUE,
    LocationID    INTEGER NOT NULL REFERENCES Location(LocationID) ON DELETE RESTRICT,
    ManagerName   TEXT NOT NULL
);
CREATE TABLE Family (
    FamilyID     INTEGER PRIMARY KEY AUTOINCREMENT,
    HeadOfFamily TEXT NOT NULL,
    ContactInfo  TEXT,
    CampID       INTEGER NOT NULL REFERENCES Camp(CampID) ON DELETE RESTRICT,
    RegisteredOn DATE NOT NULL DEFAULT CURRENT_DATE
);
CREATE TABLE Person (
    PersonID         INTEGER PRIMARY KEY AUTOINCREMENT,
    FamilyID         INTEGER NOT NULL REFERENCES Family(FamilyID) ON DELETE CASCADE,
    FullName         TEXT NOT NULL,
    Age              INTEGER NOT NULL CHECK (Age BETWEEN 0 AND 120),
    Gender           TEXT NOT NULL CHECK (Gender IN ('F','M','O')),
    MedicalCondition TEXT DEFAULT 'None'
);
CREATE TABLE Volunteer (
    VolunteerID INTEGER PRIMARY KEY AUTOINCREMENT,
    FullName    TEXT NOT NULL,
    Phone       TEXT NOT NULL UNIQUE,
    Skill       TEXT NOT NULL,
    Status      TEXT NOT NULL DEFAULT 'Available' CHECK (Status IN ('Available','Assigned','Inactive'))
);
CREATE TABLE ReliefItem (
    ItemID   INTEGER PRIMARY KEY AUTOINCREMENT,
    ItemName TEXT NOT NULL UNIQUE,
    Category TEXT NOT NULL CHECK (Category IN ('Food','Water','Shelter','Clothing','Medical')),
    Unit     TEXT NOT NULL
);
CREATE TABLE Stock (
    StockID     INTEGER PRIMARY KEY AUTOINCREMENT,
    ItemID      INTEGER NOT NULL REFERENCES ReliefItem(ItemID) ON DELETE RESTRICT,
    WarehouseID INTEGER NOT NULL REFERENCES Warehouse(WarehouseID) ON DELETE RESTRICT,
    Quantity    INTEGER NOT NULL DEFAULT 0 CHECK (Quantity >= 0),
    LastUpdated TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (ItemID, WarehouseID)
);
CREATE TABLE Donation (
    DonationID   INTEGER PRIMARY KEY AUTOINCREMENT,
    DonorName    TEXT NOT NULL,
    ItemID       INTEGER NOT NULL REFERENCES ReliefItem(ItemID) ON DELETE RESTRICT,
    WarehouseID  INTEGER NOT NULL REFERENCES Warehouse(WarehouseID) ON DELETE RESTRICT,
    Quantity     INTEGER NOT NULL CHECK (Quantity > 0),
    DonationDate DATE NOT NULL DEFAULT CURRENT_DATE
);
CREATE TABLE Need (
```

```
    NeedID   INTEGER PRIMARY KEY AUTOINCREMENT,
    FamilyID INTEGER NOT NULL REFERENCES Family(FamilyID) ON DELETE CASCADE,
    ItemID   INTEGER NOT NULL REFERENCES ReliefItem(ItemID) ON DELETE RESTRICT,
    Quantity INTEGER NOT NULL CHECK (Quantity > 0),
    Status   TEXT NOT NULL DEFAULT 'Pending' CHECK (Status IN ('Pending','Partial','Fulfilled')),
    UNIQUE (FamilyID, ItemID)
);
CREATE TABLE Distribution (
    DistributionID INTEGER PRIMARY KEY AUTOINCREMENT,
    NeedID         INTEGER NOT NULL REFERENCES Need(NeedID) ON DELETE RESTRICT,
    StockID        INTEGER NOT NULL REFERENCES Stock(StockID) ON DELETE RESTRICT,
    VolunteerID    INTEGER NOT NULL REFERENCES Volunteer(VolunteerID) ON DELETE RESTRICT,
    Quantity       INTEGER NOT NULL CHECK (Quantity > 0),
    DistDate       DATE NOT NULL DEFAULT CURRENT_DATE
);
CREATE TABLE Assignment (
    AssignmentID INTEGER PRIMARY KEY AUTOINCREMENT,
    VolunteerID  INTEGER NOT NULL REFERENCES Volunteer(VolunteerID) ON DELETE RESTRICT,
    CampID       INTEGER NOT NULL REFERENCES Camp(CampID) ON DELETE RESTRICT,
    Role         TEXT NOT NULL,
    StartDate    DATE NOT NULL,
    EndDate      DATE,
    CHECK (EndDate IS NULL OR EndDate >= StartDate)
);
CREATE TABLE Referral (
    ReferralID   INTEGER PRIMARY KEY AUTOINCREMENT,
    PersonID     INTEGER NOT NULL REFERENCES Person(PersonID) ON DELETE CASCADE,
    CampID       INTEGER NOT NULL REFERENCES Camp(CampID) ON DELETE RESTRICT,
    Facility     TEXT NOT NULL,
    Reason       TEXT NOT NULL,
    ReferralDate DATE NOT NULL DEFAULT CURRENT_DATE,
    Status       TEXT NOT NULL DEFAULT 'Open' CHECK (Status IN ('Open','Attended','Closed'))
);
-- Business-rule triggers -------------------------------------------------
CREATE TRIGGER trg_dist_validate BEFORE INSERT ON Distribution
BEGIN
    SELECT CASE WHEN (SELECT ItemID FROM Need WHERE NeedID = NEW.NeedID)
                  <> (SELECT ItemID FROM Stock WHERE StockID = NEW.StockID)
           THEN RAISE(ABORT, 'Item mismatch: stock item differs from the needed item') END;
    SELECT CASE WHEN (SELECT Status FROM Need WHERE NeedID = NEW.NeedID) = 'Fulfilled'
           THEN RAISE(ABORT, 'Duplicate entitlement: need already fulfilled') END;
    SELECT CASE WHEN NEW.Quantity > (SELECT n.Quantity - COALESCE((SELECT SUM(d.Quantity) FROM Distribution d
                                    WHERE d.NeedID = n.NeedID), 0) FROM Need n WHERE n.NeedID = NEW.NeedID)
           THEN RAISE(ABORT, 'Over-allocation: quantity exceeds remaining need') END;
    SELECT CASE WHEN NEW.Quantity > (SELECT Quantity FROM Stock WHERE StockID = NEW.StockID)
           THEN RAISE(ABORT, 'Insufficient stock in selected warehouse') END;
END;
CREATE TRIGGER trg_dist_apply AFTER INSERT ON Distribution
BEGIN
    UPDATE Stock SET Quantity = Quantity - NEW.Quantity, LastUpdated = CURRENT_TIMESTAMP
     WHERE StockID = NEW.StockID;
    UPDATE Need SET Status = CASE
        WHEN (SELECT SUM(Quantity) FROM Distribution WHERE NeedID = NEW.NeedID) >= Quantity THEN 'Fulfilled'
        ELSE 'Partial' END
     WHERE NeedID = NEW.NeedID;
END;
CREATE TRIGGER trg_donation_stock AFTER INSERT ON Donation
BEGIN
    INSERT INTO Stock (ItemID, WarehouseID, Quantity) VALUES (NEW.ItemID, NEW.WarehouseID, NEW.Quantity)
    ON CONFLICT (ItemID, WarehouseID)
    DO UPDATE SET Quantity = Quantity + NEW.Quantity, LastUpdated = CURRENT_TIMESTAMP;
END;
CREATE INDEX idx_family_camp ON Family(CampID);
CREATE INDEX idx_need_family ON Need(FamilyID);
CREATE INDEX idx_dist_need   ON Distribution(NeedID);
-- Representative sample data (illustrative)
INSERT INTO Disaster (DisasterName, DisasterType, StartDate, Severity) VALUES
 ('Manjeera Basin Floods 2026','Flood','2026-09-28','Critical'),
 ('Gajwel Cyclone Spell 2026','Cyclone','2026-10-02','High');
INSERT INTO Location (Village, District, DisasterID) VALUES
 ('Siddipet','Siddipet',1),
 ('Husnabad','Siddipet',1),
 ('Gajwel','Siddipet',2),
 ('Karimnagar','Karimnagar',1);
INSERT INTO Camp (CampName, LocationID, Capacity) VALUES
 ('Siddipet Relief Camp A',1,300),('Husnabad Camp B',2,180),('Gajwel Camp C',3,220);
INSERT INTO Warehouse (WarehouseName, LocationID, ManagerName) VALUES
 ('Siddipet Central Warehouse',1,'K. Srinivas'),('Karimnagar Depot',4,'M. Fatima');
INSERT INTO ReliefItem (ItemName, Category, Unit) VALUES
 ('Rice','Food','kg'),('Drinking Water','Water','litre'),('Blanket','Clothing','piece'),
 ('Tarpaulin Tent','Shelter','piece'),('First-Aid Kit','Medical','kit'),('Cooking Oil','Food','litre');
INSERT INTO Family (HeadOfFamily, ContactInfo, CampID, RegisteredOn) VALUES
 ('Ravi Kumar','9848011101',1,'2026-09-29'),('Lakshmi Devi','9848011102',1,'2026-09-29'),
 ('Anwar Khan','9848011103',2,'2026-09-30'),('Sunita Reddy','9848011104',2,'2026-09-30'),
 ('Mahesh Goud','9848011105',3,'2026-10-03'),('Padma Rao','9848011106',3,'2026-10-03');
INSERT INTO Person (FamilyID, FullName, Age, Gender, MedicalCondition) VALUES
 (1,'Ravi Kumar',42,'M','None'),(1,'Sridevi Kumar',38,'F','None'),(1,'Arjun Kumar',14,'M','None'),(1,'Meena
Kumar',9,'F','Asthma'),(1,'Kamala Bai',68,'F','Diabetes'),
 (2,'Lakshmi Devi',55,'F','Hypertension'),(2,'Vamsi Devi',27,'M','None'),(2,'Harini Devi',4,'F','None'),(2,'Kiran
Devi',30,'F','None'),
 (3,'Anwar Khan',47,'M','None'),(3,'Shabana Khan',44,'F','None'),(3,'Imran Khan',20,'M','None'),(3,'Zoya
Khan',16,'F','None'),(3,'Aliya Khan',8,'F','None'),(3,'Salma Bi',72,'F','Arthritis'),
 (4,'Sunita Reddy',35,'F','None'),(4,'Teja Reddy',11,'M','None'),(4,'Nandini Reddy',2,'F','None'),
```

```
 (5,'Mahesh Goud',50,'M','None'),(5,'Yadamma Goud',46,'F','None'),(5,'Bhaskar Goud',22,'M','None'),(5,'Ramya
Goud',19,'F','None'),(5,'Chandu Goud',12,'M','None'),(5,'Pooja Goud',6,'F','None'),(5,'Narsamma',75,'F','Heart
condition'),
 (6,'Padma Rao',63,'F','Diabetes'),(6,'Sai Rao',31,'M','None');
INSERT INTO Volunteer (FullName, Phone, Skill, Status) VALUES
 ('Anitha Rao','9000000001','Logistics','Assigned'),('Imran Shaikh','9000000002','Driver','Assigned'),
 ('Suresh Babu','9000000003','Medical Aid','Assigned'),('Divya Nair','9000000004','Registration','Assigned'),
 ('Karthik Yadav','9000000005','Cooking','Available');
INSERT INTO Assignment (VolunteerID, CampID, Role, StartDate, EndDate) VALUES
 (1,1,'Stores in-charge','2026-09-29',NULL),(2,1,'Transport','2026-09-29',NULL),
 (3,2,'First-aid desk','2026-09-30',NULL),(4,2,'Registration desk','2026-09-30','2026-10-10'),
 (4,3,'Registration desk','2026-10-11',NULL),(1,3,'Stores in-charge','2026-10-03','2026-10-08');
-- Opening stock (before donations)
INSERT INTO Stock (ItemID, WarehouseID, Quantity) VALUES
 (1,1,400),(2,1,900),(3,1,40),(4,2,12),(5,2,8),(1,2,150),(3,2,9),(6,1,60);
INSERT INTO Donation (DonorName, ItemID, WarehouseID, Quantity, DonationDate) VALUES
 ('Rotary Club Siddipet',1,1,100,'2026-10-05'),('Telangana Red Cross',5,2,20,'2026-10-06'),
 ('Hyderabad Textile Assn.',3,1,60,'2026-10-07'),('Anonymous',2,1,300,'2026-10-08'),('Lions Club
Karimnagar',4,2,10,'2026-10-09');
INSERT INTO Need (FamilyID, ItemID, Quantity) VALUES
 (1,1,50),(1,3,5),(2,2,80),(3,4,1),(3,1,60),(4,5,1),(5,2,100),(5,3,7),(6,1,20),(2,3,4),(4,6,5),(6,5,1);
-- Distributions (validated and applied by triggers)
INSERT INTO Distribution (NeedID, StockID, VolunteerID, Quantity, DistDate) VALUES
 (1,1,1,50,'2026-10-09'),(2,3,1,2,'2026-10-10'),(5,6,2,20,'2026-10-10'),(7,2,2,100,'2026-10-11'),(9,1,1,20,'2026-
10-11');
INSERT INTO Referral (PersonID, CampID, Facility, Reason, ReferralDate) VALUES
 (4,1,'Area Hospital Siddipet','Asthma attack after flooding','2026-10-04'),
 (15,2,'PHC Husnabad','Joint pain, mobility support','2026-10-05'),
 (25,3,'District Hospital Gajwel','Cardiac check-up','2026-10-06');
```

APPENDIX B - APPLICATION SOURCE CODE

The source code is supplied as a project folder with the files below. Table 10 holds the repository link.

```
relief_app.py  - Python application: CRUD functions, transactions and six
reports
schema.sql     - DDL: 14 tables, constraints, triggers, indexes
data.sql       - DML: sample data
run_tests.py   - executes the 13 test cases and records the results
relief.db      - SQLite database created from schema.sql and data.sql
```

```
Run:  python3 relief_app.py [camp_occupancy|unmet_needs|stock_alerts|
                             audit_trail|volunteer_load|no_aid]
      python3 run_tests.py
```

A browser-based prototype of the distribution screen, which demonstrates the same business rules with in-memory sample data (not connected to the database), is available at: https://claude.ai/artifact/2unqyF8UUfZQFuMZ7nxDF3

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Table 15 (Section 3.8) holds the full results of the 13 test cases, which running run_tests.py reproduces and saves to results.json. No further test data or outputs are attached.